The Client & Server

SWE 4537: Server Programming
Lecture 01

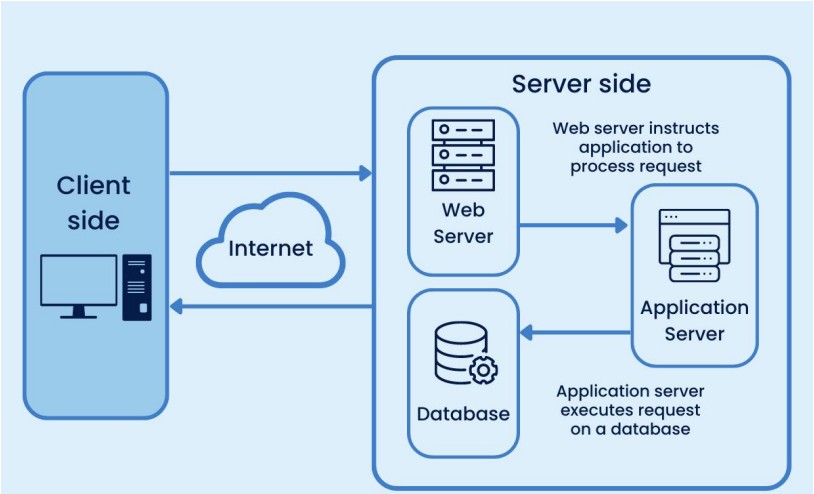
Ishmam Tashdeed | Asaduzzaman Herok
Department of CSE
Islamic University of Technology

October 29, 2026

Lecture Overview

1. The Client Server
2. Digital Address
3. Language / Protocol of Communication
4. End

The Client Server



Web Client

Web Client

A Web Client is the software on your device (like your computer, phone, or tablet) that you use to access the web.

What it does:

- Takes the web address (URL) you type in (e.g., https://www.google.com).
- Sends an HTTP request to the correct server, asking for the content of that page.
- Receives the response from the server (which can be code like HTML, CSS, and JavaScript, or files like JSON).
- Interprets and renders this code into the visual webpage you see and interact with.

Examples:

Any browser (Chrome, Edge), Application software that connects to internet.

Web Server



Web Server (Part I)

Web Server

A Web Server is the system that stores, processes, and delivers web content to clients. It can refer two things:

- **Hardware**: The physical computer (or virtual machine) that stores all the website's data and Web applications.
- **Software**: The application that runs on the hardware. Its main job is to understand URLs and the Hypertext Transfer Protocol (HTTP).

Web Server (Part II)

What it does:

- **Receives Requests**: server is always "on" and "listening" for incoming requests from clients
- **Processes Requests**: depends on what the client asked for
  - **For a Static File**: server just finds that specific file
  - **For a Dynamic Request**: It runs the web application's backend code.
- **Sends a Response**: Upon finding file or generated the data, it packages it up and sends it back to the client (your browser) as an HTTP response along with a status code.

Web Server (Part III)

Static vs Dynamic Server

- **Static Web Server**: simply sends files as is to the client.
- **Dynamic Web Server**: works with other components (like an application server and a database) to create content on the fly. The content can be personalized for the user,

Examples:

- Apache HTTP Server
- Nginx (pronounced “Engine-X”)
- Microsoft IIS (Internet Information Services)

Web Application (Web App)

Web Application

A dynamic, interactive program that can be accessed from a browser. It runs in both place client side and server side by parts. Many users access the same deployed application, but each may have different sessions, permissions, and data.

- **Client-Side (Frontend)**: This is the part that runs in your browser. Responsible for the user interface (UI) you see and click on.
- **Server-Side (Backend)**: This is the part that runs on the web server, contains the "business logic".

Common Example:

Gmail, Outlook 365, Google Workspace (Docs, Sheets), Microsoft Office Online, Facebook, X (Twitter), Instagram.

IP & Port

IP (Internet Protocol) address

A unique numerical label assigned to every device (like a server or your computer) on a network. Its single purpose is identification in a network. Example: $172.217.14.228$

Port Number

A port number is a 16-bit number (from 0 to 65535) used to identify a specific application or service on a server machine. Ports are split into two main types:

- Server Ports (Well-Known Ports): standardized and predictable. Server listens on a specific, well-known port for incoming requests. Example: Port 80 (HTTP), Port 443 (HTTPS)
- Client Ports (Temporary Ports): Private ports that the client opens for itself. Generally a high value number.

The Full Address (Part I)

Socket

An IP address and port identify a network endpoint. A socket is the software endpoint used for network communication. Example: $192.168.1.10 : 51255$

Uniform Resource Locator (URL)

The human-readable address that ties everything together. It's the single instruction a client uses to find the right server.

The Full Address (Part II)

URL Deconstruction

`https://www.example.com:8080/products?id=123`

`https://en.wikipedia.org/wiki/Web%20server#Hardware`

- `https://` : The protocol/language for client server conversation.
- `www.example.com` : Hostname/Domain. The human-friendly name for the server.
- `:8080` : Port number.
- `/products` : URL path. Specifies the resource client want.
- `?id=123` : Query parameter.
- `%20` : URL encoding.
- `# Hardware` : (Fragment locator) Optional final part.

HTTP Protocol

HTTP (Hypertext Transfer Protocol)

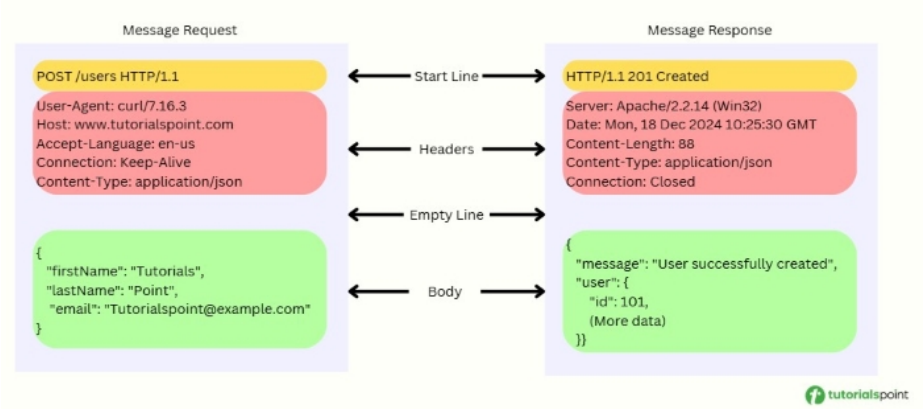
It is the set of rules that web clients and web servers use to communicate with each other.

Basic Features

- **Stateless**: Each request is treated as a new request i.e both client and server forget about each other after completion of a request unless cookies, tokens or sessions are used.
- **Request–Response Model**: A client sends a request, and the server returns a response. The underlying connection may be reused for multiple exchanges.
- **HTTP Methods**: HTTP defines various methods for different actions such as GET, POST, PUT and more.
- **Caching Support**: HTTP provides support for caching which improves the performance by storing the copies of responses and reusing them later.

The HTTP Message

An HTTP message can be a **request** sent by the client to the server or a **response** sent by the server to the client.



HTTP Request Message (Part I)

Has following four components

- **A Request-line**: <Method> <Request-URI> <HTTP-Version>
- **Header** Zero or more header fields followed by CRLF
- **An empty line** (i.e., a line with nothing preceding the CRLF) indicating the end of the header fields
- **Message body**: Optional

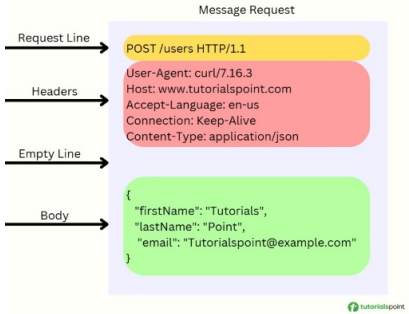


Figure: Caption

HTTP Request Message (Part II)

Request Methods

The request method indicates the method to be performed on the resource identified by the given Request-URI.

- **GET**: The GET method is used to retrieve information from the given server using a given URI.
- **POST**: A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.
- **PUT**: Replaces all the current representations of the target resource with the uploaded content.
- **DELETE**: Removes all the current representations of the target resource given by URI.

HTTP Request Message (Part III)

Request Headers

Allow the client to pass additional information about the request, and about the client itself, to the server.

Message Body

The content sent to the server. For get method this body is empty. For Post, PUT method it contains data.

HTTP Response Message (Part I)

Has following four components

- **A Status-line**: <HTTP-Version> <Status-Code> <Reason-Phrase>
- **Header** Zero or more header fields followed by CRLF
- **An empty line** (i.e., a line with nothing preceding the CRLF) indicating the end of the header fields
- **Message body**: Optional

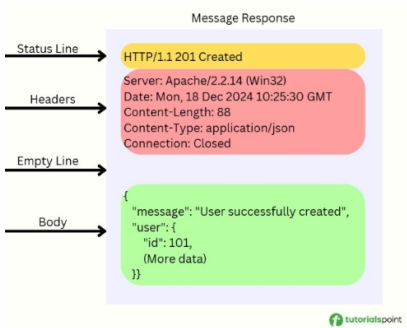


Figure: Caption

HTTP Response Message (Part II)

Status Code

A 3-digit integer. First digit defines the class of response. Last have no categorization role.

- **1xx: Informational**: The request was received and the process is continuing.
- **2xx: Success**: The action was successfully received, understood, and accepted.
- **3xx: Redirection**: Further action must be taken in order to complete the request.
- **4xx: Client Error**: The request contains incorrect syntax or cannot be fulfilled.
- **5xx: Server Error**: The server failed to fulfill an apparently valid request.

Response Header

Allow the server to pass additional information about the response which cannot be placed in the Status- Line. These header fields give information about the server

🤝 Conclusion & Q&A

Thank You!

👏 Thank you for your time and attention. 👏

🗨️ Any questions or clarifications. 💭